Praktikum 3

NAMA	: Ihsanta Zaki Sanjaya
KELAS	: 2 D4 Teknik Komputer B
NRP	: 3223600058
DOSEN	: Prof. Ir. Dadet Pramadihanto, M.Eng, Ph.D
MATA KULIAH	: Praktikum Pengolahan Sinyal dan Citra
TANGGAL	: Kamis, 13 Maret 2025

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2025

Praktikum 3
Discrete-Time Fourier Transform

1. DASAR TEORI

DTFT merupakan bentuk transformasi Fourier yang dirancang khusus untuk sinyal yang bersifat diskrit dan tidak periodik. Berbeda dengan Discrete Fourier Transform (DFT), yang hanya memberikan informasi pada titik-titik tertentu di domain frekuensi, DTFT menghasilkan spektrum yang lebih halus dan kontinu. Dengan kata lain, DTFT membantu kita memahami bagaimana komponen frekuensi suatu sinyal tersebar, sehingga kita bisa melihat bagaimana sinyal tersebut dapat dimodifikasi atau diproses lebih lanjut.

DTFT dari suatu sinyal diskrit x[n] dinyatakan dalam bentuk persamaan sebagai berikut:

$$X(e^{j\omega}) = \sum_{n=-\infty}^{\infty} x[n]e^{-j\omega n}$$

X(ejω) merupakan representasi sinyal dalam domain frekuensi, sedangkan ω adalah frekuensi sudut dalam radian. Konsep ini sangat berguna dalam berbagai bidang seperti pengolahan sinyal digital, komunikasi, dan sistem kendali, di mana kita sering perlu menganalisis dan memodifikasi sinyal dalam domain frekuensi.

Salah satu sifat dasar DTFT adalah linearitas. Sifat ini menyatakan bahwa jika kita memiliki dua sinyal berbeda, misalnya x1[n] dan x2[n], lalu mengombinasikannya dengan faktor skalar a dan b, maka hasil DTFT dari kombinasi tersebut juga merupakan kombinasi dari DTFT masing-masing sinyal, yaitu:

$$ax_1[n] + bx_2[n] \quad \longleftrightarrow \quad aX_1(e^{j\omega}) + bX_2(e^{j\omega})$$

Artinya, penjumlahan dan perkalian skalar yang dilakukan dalam domain waktu tetap berlaku dalam domain frekuensi. Sifat ini sangat berguna dalam banyak aplikasi, seperti penyaringan sinyal dan pemrosesan data audio. Selain linearitas, DTFT juga memiliki sifat pergeseran waktu. Jika suatu sinyal x[n] mengalami pergeseran sejauh n0 dalam domain waktu (misalnya, sinyal dipindahkan beberapa langkah ke kanan atau ke kiri), maka hasilnya akan tetap sama tetapi dikalikan dengan faktor eksponensial kompleks:

$$X(e^{j\omega})e^{-j\omega n_0}$$

Ini penting dalam berbagai sistem komunikasi, karena sinyal sering kali mengalami pergeseran waktu akibat keterlambatan dalam transmisi. Selanjutnya terdapat teorema konvolusi dalam DTFT yang menyatakan bahwa konvolusi dua sinyal dalam domain waktu setara dengan perkalian spektrum mereka dalam domain frekuensi:

$$x_1[n] * x_2[n] \quad \longleftrightarrow \quad X_1(e^{j\omega})X_2(e^{j\omega})$$

Konvolusi dalam domain waktu sering digunakan dalam pemrosesan sinyal, seperti pada sistem filter digital. Dengan memahami teorema ini, kita dapat menyederhanakan analisis sistem dengan memindahkan operasi dari domain waktu ke domain frekuensi, di mana konvolusi berubah menjadi perkalian yang lebih mudah dihitung. Untuk mengubah kembali sinyal yang telah ditransformasikan ke domain frekuensi kembali ke domain waktu, digunakan Inverse DTFT yang dinyatakan dengan persamaan:

$$x[n] = \frac{1}{2\pi} \int_{-\pi}^{\pi} X(e^{j\omega}) e^{j\omega n} d\omega$$

Inverse DTFT berguna dalam rekonstruksi sinyal, di mana sinyal asli dapat diperoleh kembali dari representasi frekuensinya. Dalam aplikasi praktis, metode numerik digunakan untuk mendekati integral ini. Tak hanya itu, DTFT juga memiliki penerapan khusus pada sinyal periodik. Untuk sinyal yang memiliki periode N, spektrum frekuensinya akan terbatas hanya pada frekuensi tertentu, menghasilkan deretan puncak dalam domain frekuensi. Hal ini dapat dijelaskan menggunakan konsep Fourier Series dalam bentuk spektrum diskrit yang muncul pada frekuensi kelipatan dasar dari sinyal periodik tersebut.

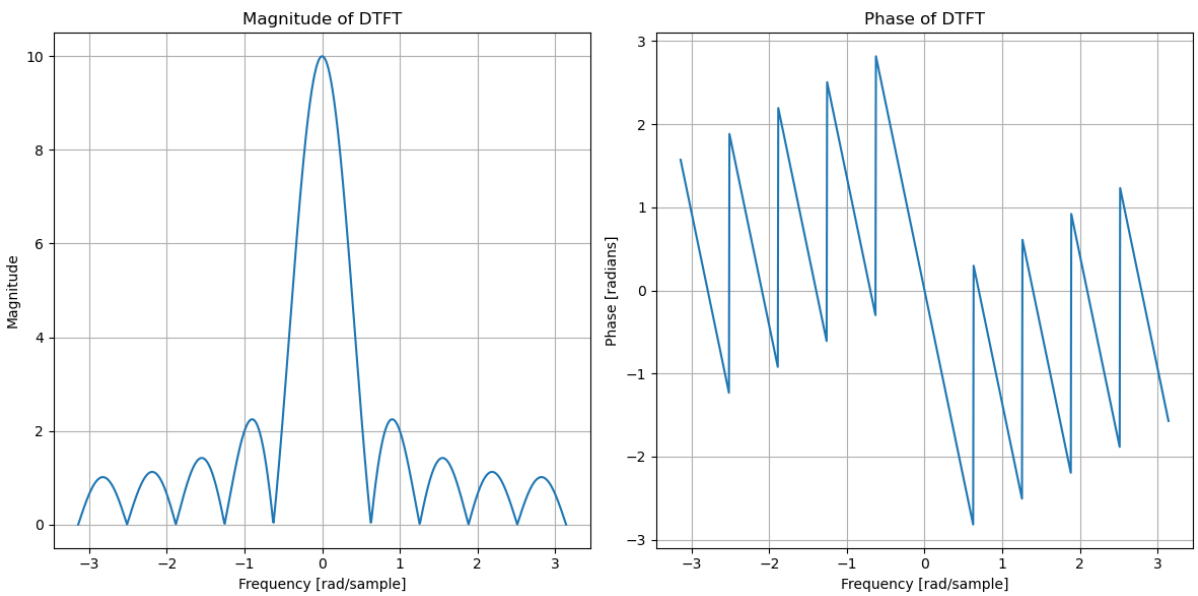
2. PROSEDUR

2.4.1

Program:

```
import numpy as np
import matplotlib.pyplot as plt

# Define the signal x[n]
N = 10  # Length of the rectangular pulse
x = np.ones(N)

# Compute the DTFT using a large number of frequency points
omega = np.linspace(-np.pi, np.pi, 1000)  # Frequency range
X = np.zeros_like(omega, dtype=complex)

# Compute the DTFT by evaluating the sum for each frequency
for i, w in enumerate(omega):
    X[i] = np.sum(x * np.exp(-1j * w * np.arange(N)))

# Plot the magnitude and phase of the DTFT
plt.figure(figsize=(12, 6))

# Magnitude plot
plt.subplot(1, 2, 1)
plt.plot(omega, np.abs(X))
plt.title("Magnitude of DTFT")
plt.xlabel("Frequency [rad/sample]")
plt.ylabel("Magnitude")
plt.grid(True)

# Phase plot
plt.subplot(1, 2, 2)
plt.plot(omega, np.angle(X))
plt.title("Phase of DTFT")
plt.xlabel("Frequency [rad/sample]")
plt.ylabel("Phase [radians]")
plt.grid(True)

plt.tight_layout()
plt.show()
```

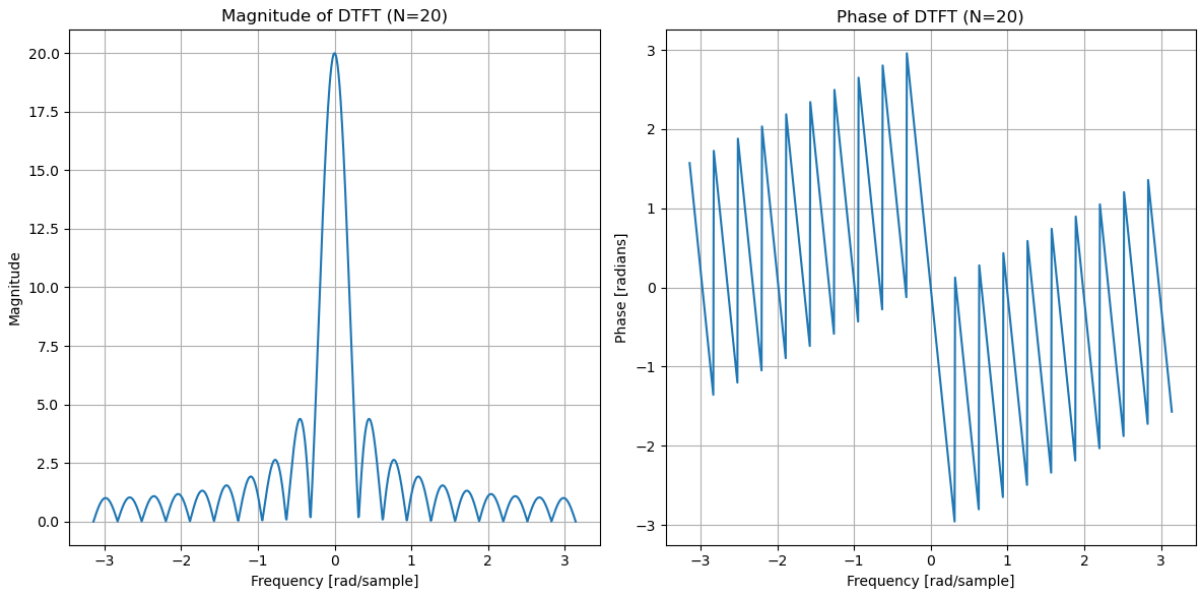
Hasil:



Variasi 1, mengubah panjang pulse (N) menjadi 20:

```
import numpy as np
import matplotlib.pyplot as plt

# Define the signal x[n] with a longer rectangular pulse
N = 20  # Increased length
x = np.ones(N)

# Compute the DTFT using a large number of frequency points
omega = np.linspace(-np.pi, np.pi, 1000)
X = np.zeros_like(omega, dtype=complex)

# Compute the DTFT
for i, w in enumerate(omega):
    X[i] = np.sum(x * np.exp(-1j * w * np.arange(N)))

# Plot the magnitude and phase of the DTFT
plt.figure(figsize=(12, 6))

# Magnitude plot
plt.subplot(1, 2, 1)
plt.plot(omega, np.abs(X))
plt.title("Magnitude of DTFT (N=20)")
plt.xlabel("Frequency [rad/sample]")
plt.ylabel("Magnitude")
plt.grid(True)

# Phase plot
plt.subplot(1, 2, 2)
plt.plot(omega, np.angle(X))
plt.title("Phase of DTFT (N=20)")
plt.xlabel("Frequency [rad/sample]")
plt.ylabel("Phase [radians]")
plt.grid(True)

plt.tight_layout()
plt.show()
```

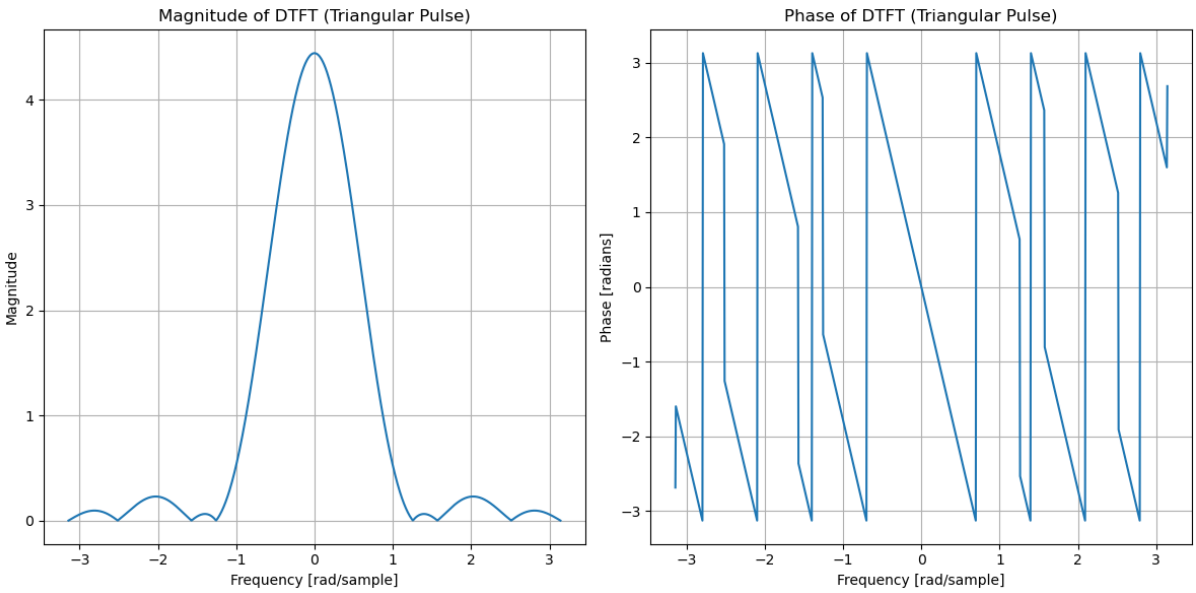
Hasil:



Variasi 2, menggunakan *triangle pulse*:

```
import numpy as np
import matplotlib.pyplot as plt

# Define a triangular pulse signal
N = 10  # Length of the signal
x = np.bartlett(N)  # Bartlett window creates a triangular shape

# Compute the DTFT using a large number of frequency points
omega = np.linspace(-np.pi, np.pi, 1000)
X = np.zeros_like(omega, dtype=complex)

# Compute the DTFT
for i, w in enumerate(omega):
    X[i] = np.sum(x * np.exp(-1j * w * np.arange(N)))

# Plot the magnitude and phase of the DTFT
plt.figure(figsize=(12, 6))

# Magnitude plot
plt.subplot(1, 2, 1)
plt.plot(omega, np.abs(X))
plt.title("Magnitude of DTFT (Triangular Pulse)")
plt.xlabel("Frequency [rad/sample]")
plt.ylabel("Magnitude")
plt.grid(True)

# Phase plot
plt.subplot(1, 2, 2)
plt.plot(omega, np.angle(X))
plt.title("Phase of DTFT (Triangular Pulse)")
plt.xlabel("Frequency [rad/sample]")
plt.ylabel("Phase [radians]")
plt.grid(True)

plt.tight_layout()
plt.show()
```

Hasil:

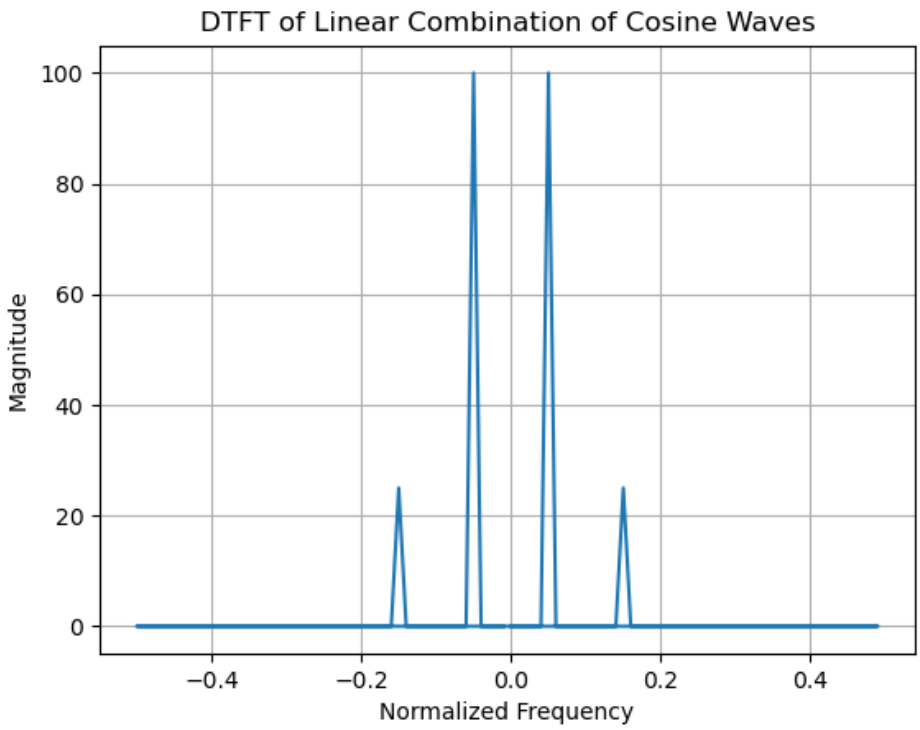


2.6.1

Program:

```
import numpy as np
import matplotlib.pyplot as plt

# Define two signals
n = np.arange(0, 100, 1)
x1 = np.cos(0.1 * np.pi * n)  # First signal: cosine wave
x2 = np.cos(0.3 * np.pi * n)  # Second signal: cosine wave

# Linear combination: a * x1 + b * x2
a = 2
b = 0.5
x = a * x1 + b * x2

# Compute the DTFT using NumPy's FFT
X = np.fft.fft(x)
freqs = np.fft.fftfreq(len(n))

# Plot the magnitude spectrum
plt.plot(freqs, np.abs(X))
plt.title("DTFT of Linear Combination of Cosine Waves")
plt.xlabel("Normalized Frequency")
plt.ylabel("Magnitude")
plt.grid(True)
plt.show()
```

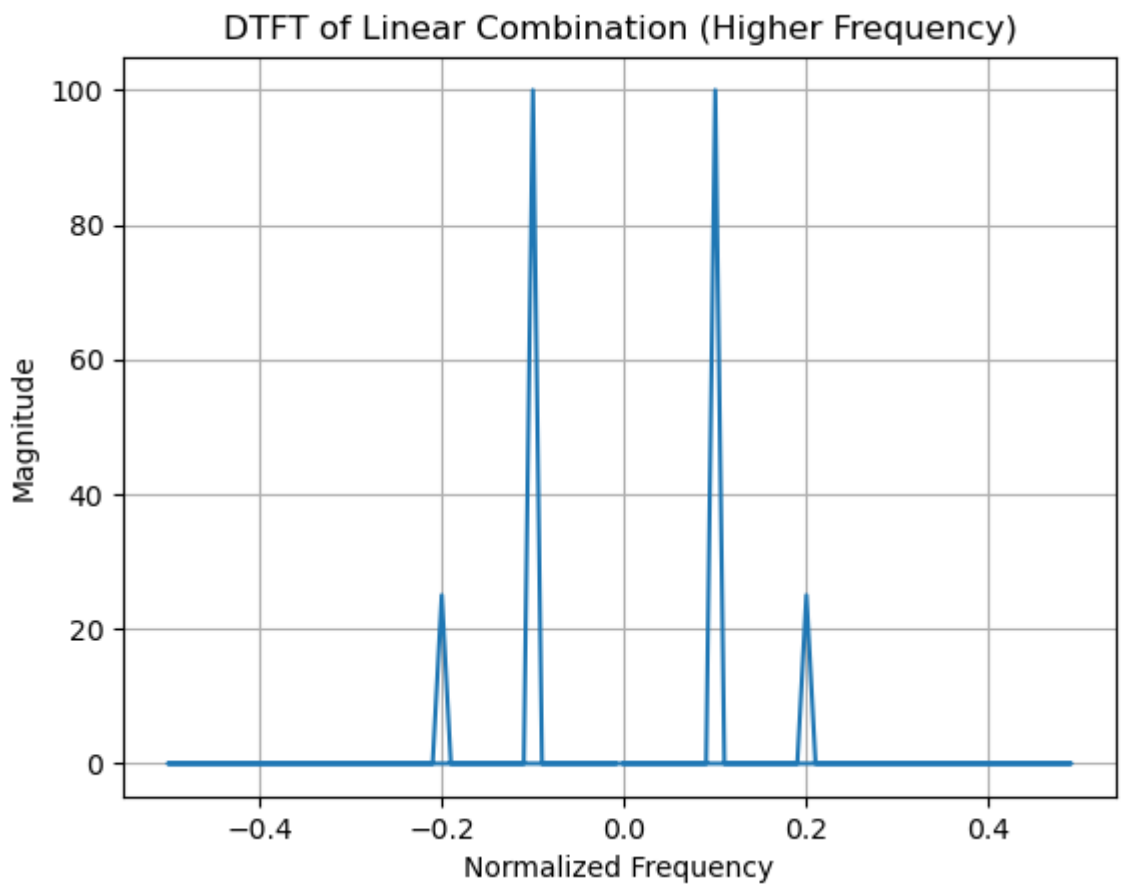
Hasil:



Variasi 1, mengubah frekuensi sinyal x1 dan x2:

```
import numpy as np
import matplotlib.pyplot as plt

# Define two signals with higher frequencies
n = np.arange(0, 100, 1)
x1 = np.cos(0.2 * np.pi * n)  # First signal: higher frequency cosine wave
x2 = np.cos(0.4 * np.pi * n)  # Second signal: higher frequency cosine wave

# Linear combination: a * x1 + b * x2
a = 2
b = 0.5
x = a * x1 + b * x2

# Compute the DTFT using NumPy's FFT
X = np.fft.fft(x)
freqs = np.fft.fftfreq(len(n))

# Plot the magnitude spectrum
plt.plot(freqs, np.abs(X))
plt.title("DTFT of Linear Combination (Higher Frequency)")
plt.xlabel("Normalized Frequency")
plt.ylabel("Magnitude")
plt.grid(True)
plt.show()
```

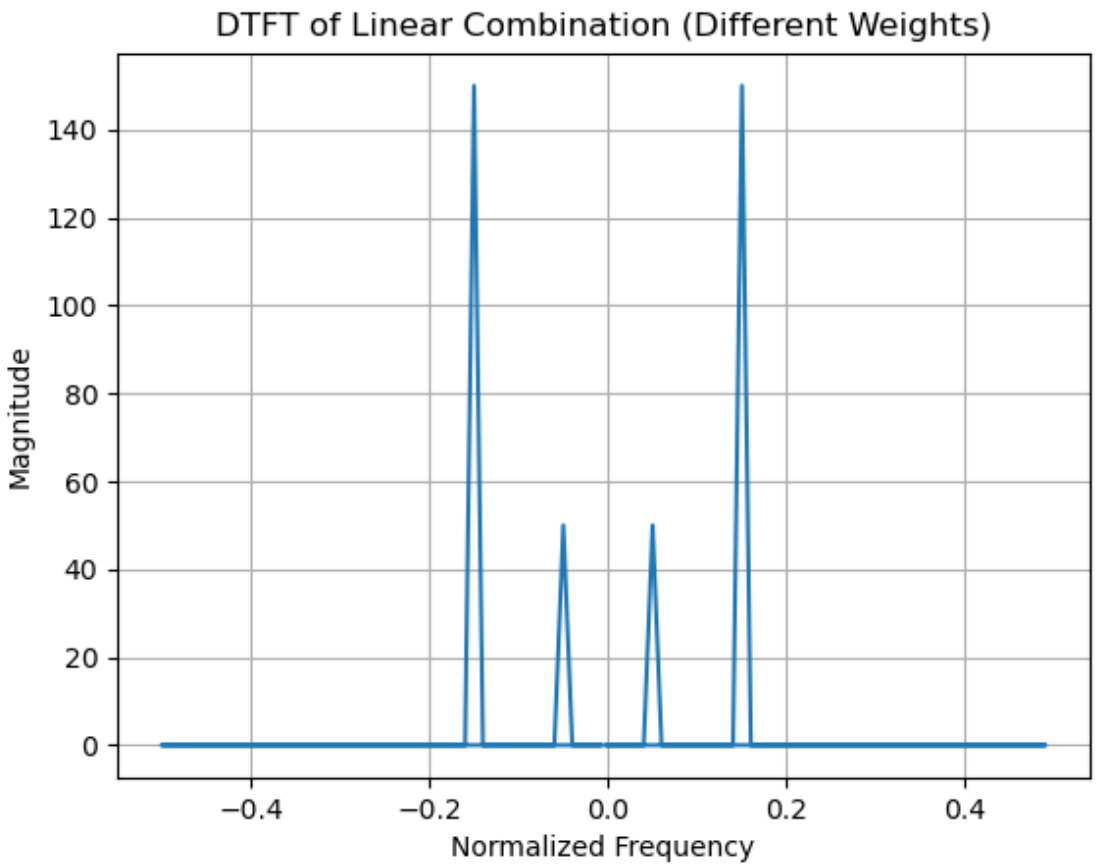
Hasil:



Variasi 2, mengubah koefisien linear kombinasi a dan b:

```
import numpy as np
import matplotlib.pyplot as plt

# Define two signals
n = np.arange(0, 100, 1)
x1 = np.cos(0.1 * np.pi * n)  # First signal: cosine wave
x2 = np.cos(0.3 * np.pi * n)  # Second signal: cosine wave

# Linear combination with different coefficients
a = 1  # Reduce weight of x1
b = 3  # Increase weight of x2
x = a * x1 + b * x2

# Compute the DTFT using NumPy's FFT
X = np.fft.fft(x)
freqs = np.fft.fftfreq(len(n))

# Plot the magnitude spectrum
plt.plot(freqs, np.abs(X))
plt.title("DTFT of Linear Combination (Different Weights)")
plt.xlabel("Normalized Frequency")
plt.ylabel("Magnitude")
plt.grid(True)
plt.show()
```

Hasil:

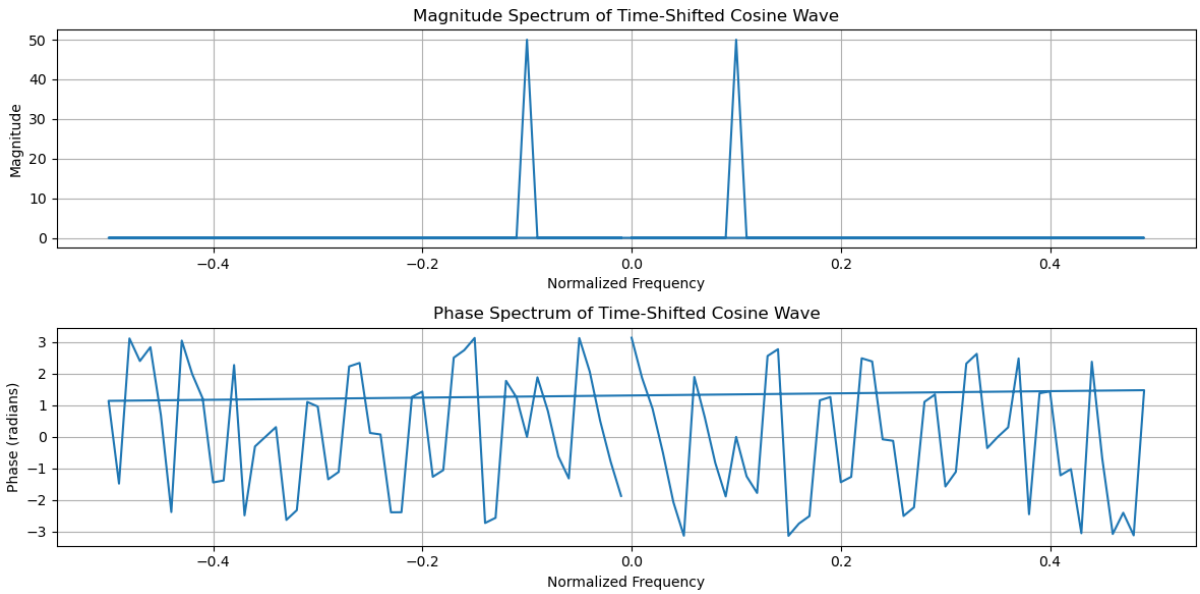


2.6.2

Program:

```
import numpy as np
import matplotlib.pyplot as plt

# Define a cosine wave
n = np.arange(0, 100, 1)
x = np.cos(0.2 * np.pi * n)

# Time shift the signal by 20 samples
n0 = 20
x_shifted = np.roll(x, n0)  # Shift the signal

# Compute the DTFT using FFT
X_shifted = np.fft.fft(x_shifted)
freqs = np.fft.fftfreq(len(n))

# Plot the magnitude and phase of the DTFT
plt.figure(figsize=(12, 6))
plt.subplot(2, 1, 1)
plt.plot(freqs, np.abs(X_shifted))
plt.title("Magnitude Spectrum of Time-Shifted Cosine Wave")
plt.xlabel("Normalized Frequency")
plt.ylabel("Magnitude")
plt.grid(True)

plt.subplot(2, 1, 2)
plt.plot(freqs, np.angle(X_shifted))
plt.title("Phase Spectrum of Time-Shifted Cosine Wave")
plt.xlabel("Normalized Frequency")
plt.ylabel("Phase (radians)")
plt.grid(True)

plt.tight_layout()
plt.show()
```

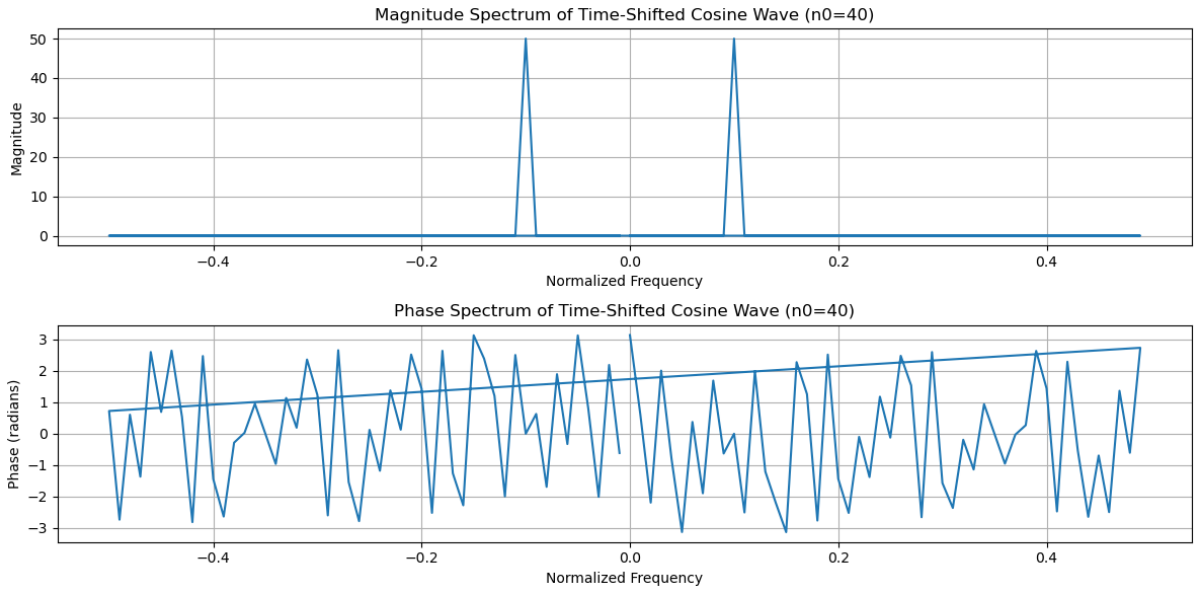
Hasil:



Variasi 1, mengubah besar pergesaran n0:

```
import numpy as np
import matplotlib.pyplot as plt

# Define a cosine wave
n = np.arange(0, 100, 1)
x = np.cos(0.2 * np.pi * n)

# Time shift the signal by 40 samples (larger shift)
n0 = 40
x_shifted = np.roll(x, n0)  # Shift the signal

# Compute the DTFT using FFT
X_shifted = np.fft.fft(x_shifted)
freqs = np.fft.fftfreq(len(n))

# Plot the magnitude and phase of the DTFT
plt.figure(figsize=(12, 6))
plt.subplot(2, 1, 1)
plt.plot(freqs, np.abs(X_shifted))
plt.title("Magnitude Spectrum of Time-Shifted Cosine Wave (n0=40)")
plt.xlabel("Normalized Frequency")
plt.ylabel("Magnitude")
plt.grid(True)

plt.subplot(2, 1, 2)
plt.plot(freqs, np.angle(X_shifted))
plt.title("Phase Spectrum of Time-Shifted Cosine Wave (n0=40)")
plt.xlabel("Normalized Frequency")
plt.ylabel("Phase (radians)")
plt.grid(True)

plt.tight_layout()
plt.show()
```

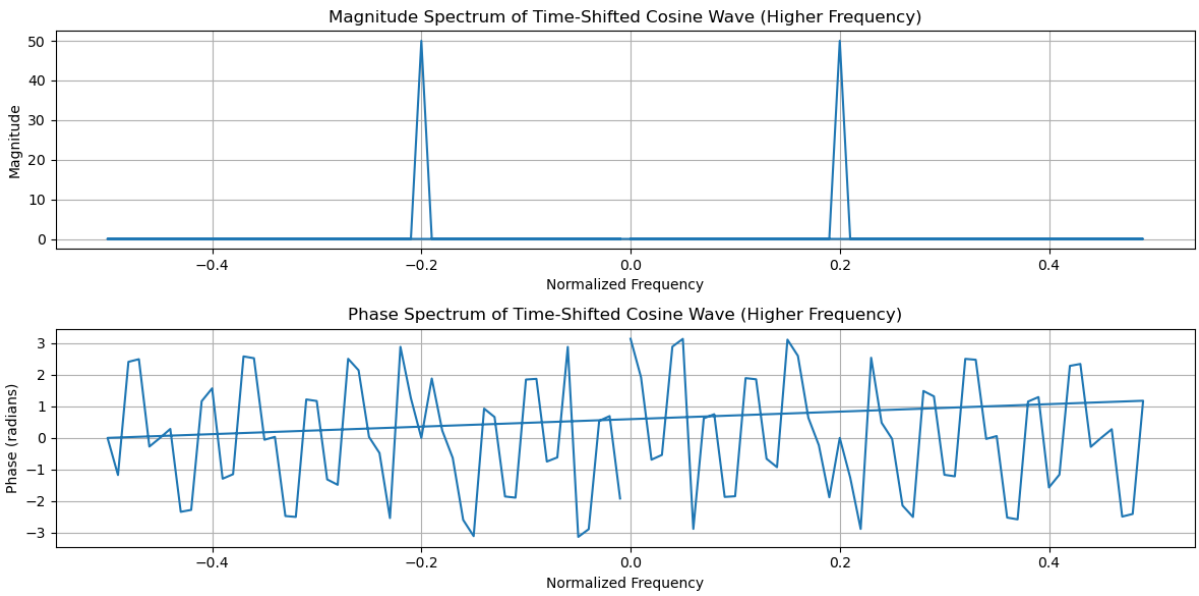
Hasil:



Variasi 2, menggunakan frekuensi sinyal yang berbeda (x):

```
import numpy as np
import matplotlib.pyplot as plt

# Define a higher frequency cosine wave
n = np.arange(0, 100, 1)
x = np.cos(0.4 * np.pi * n)  # Higher frequency

# Time shift the signal by 20 samples
n0 = 20
x_shifted = np.roll(x, n0)  # Shift the signal

# Compute the DTFT using FFT
X_shifted = np.fft.fft(x_shifted)
freqs = np.fft.fftfreq(len(n))

# Plot the magnitude and phase of the DTFT
plt.figure(figsize=(12, 6))
plt.subplot(2, 1, 1)
plt.plot(freqs, np.abs(X_shifted))
plt.title("Magnitude Spectrum of Time-Shifted Cosine Wave (Higher Frequency)")
plt.xlabel("Normalized Frequency")
plt.ylabel("Magnitude")
plt.grid(True)

plt.subplot(2, 1, 2)
plt.plot(freqs, np.angle(X_shifted))
plt.title("Phase Spectrum of Time-Shifted Cosine Wave (Higher Frequency)")
plt.xlabel("Normalized Frequency")
plt.ylabel("Phase (radians)")
plt.grid(True)

plt.tight_layout()
plt.show()
```

Hasil:

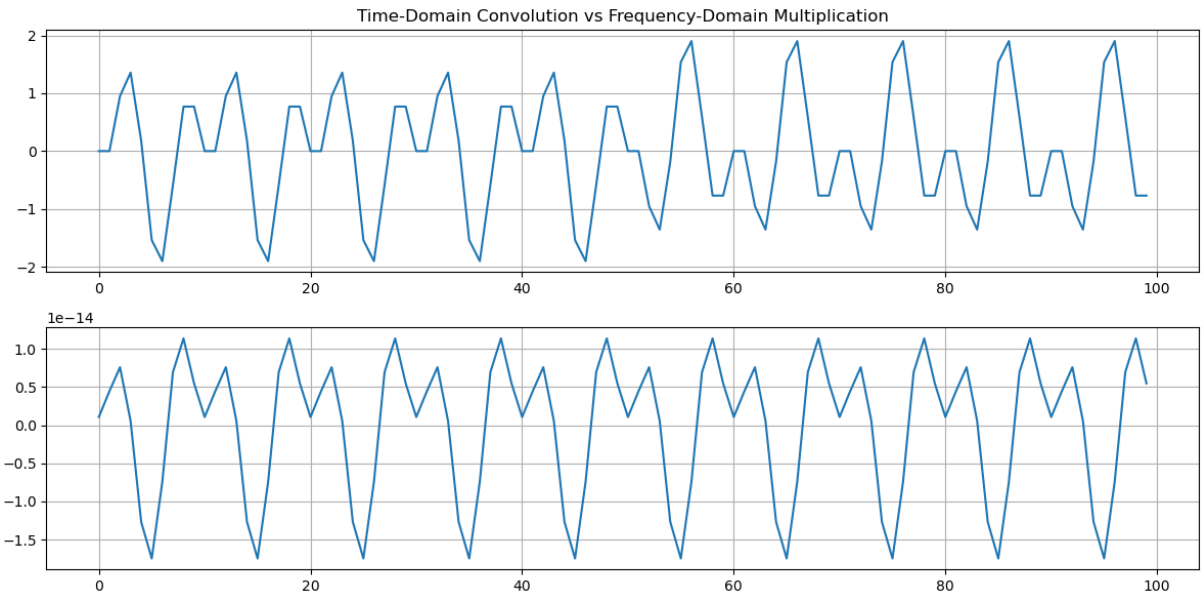


2.6.3

Program:

```
import numpy as np
import matplotlib.pyplot as plt

# Define two signals
n = np.arange(0, 100, 1)
x1 = np.cos(0.2 * np.pi * n)
x2 = np.sin(0.4 * np.pi * n)

# Time-domain convolution
x_conv = np.convolve(x1, x2, mode='same')

# Frequency-domain multiplication
X1 = np.fft.fft(x1)
X2 = np.fft.fft(x2)
X_product = X1 * X2

# Inverse FFT to get the time-domain result from frequency domain
x_freq_conv = np.fft.ifft(X_product)

# Plot the results
plt.figure(figsize=(12, 6))
plt.subplot(2, 1, 1)
plt.plot(n, x_conv, label="Time-Domain Convolution")
plt.title("Time-Domain Convolution vs Frequency-Domain Multiplication")
plt.grid(True)

plt.subplot(2, 1, 2)
plt.plot(n, np.real(x_freq_conv), label="Frequency-Domain Convolution")
plt.grid(True)

plt.tight_layout()
plt.show()
```

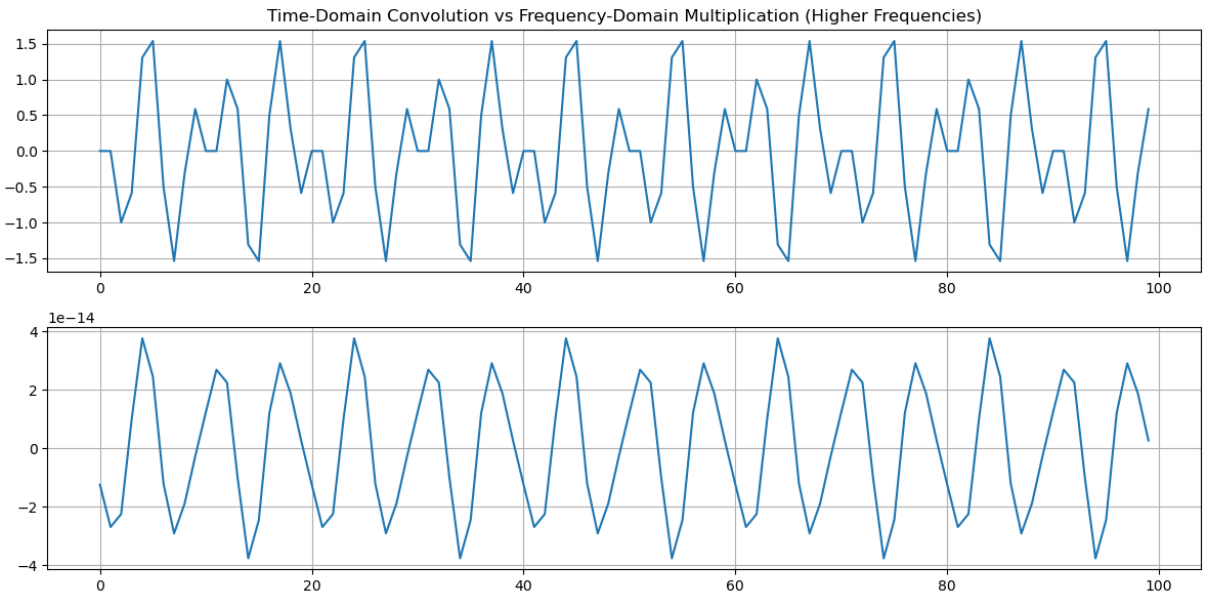
Hasil:



Variasi 1, menggunakan frekuensi yang berbeda:

```
import numpy as np
import matplotlib.pyplot as plt

# Define two signals with different frequencies
n = np.arange(0, 100, 1)
x1 = np.cos(0.3 * np.pi * n)  # Higher frequency
x2 = np.sin(0.5 * np.pi * n)  # Higher frequency

# Time-domain convolution
x_conv = np.convolve(x1, x2, mode='same')

# Frequency-domain multiplication
X1 = np.fft.fft(x1)
X2 = np.fft.fft(x2)
X_product = X1 * X2

# Inverse FFT to get the time-domain result
x_freq_conv = np.fft.ifft(X_product)

# Plot the results
plt.figure(figsize=(12, 6))
plt.subplot(2, 1, 1)
plt.plot(n, x_conv, label="Time-Domain Convolution")
plt.title("Time-Domain Convolution vs Frequency-Domain Multiplication (Higher Frequenc
plt.grid(True)

plt.subplot(2, 1, 2)
plt.plot(n, np.real(x_freq_conv), label="Frequency-Domain Convolution")
plt.grid(True)

plt.tight_layout()
plt.show()
```

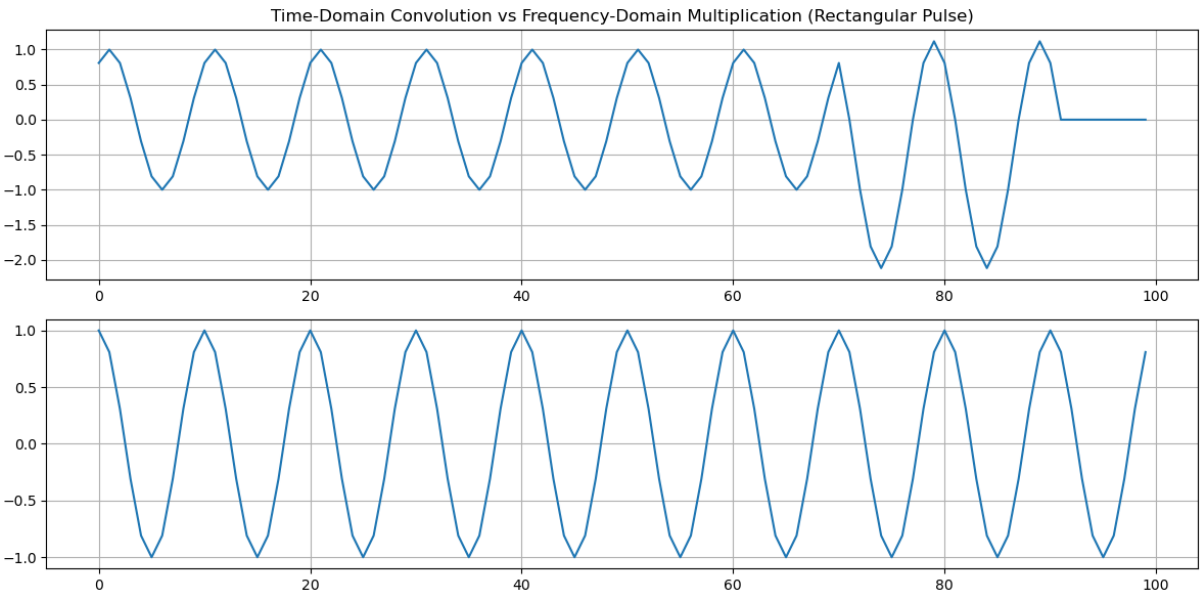
Hasil:



Variasi 2, menggunakan rectangular pulse:

```
import numpy as np
import matplotlib.pyplot as plt

# Define a cosine wave and a rectangular pulse
n = np.arange(0, 100, 1)
x1 = np.cos(0.2 * np.pi * n)  # Cosine wave
x2 = np.where((n >= 20) & (n <= 40), 1, 0)  # Rectangular pulse

# Time-domain convolution
x_conv = np.convolve(x1, x2, mode='same')

# Frequency-domain multiplication
X1 = np.fft.fft(x1)
X2 = np.fft.fft(x2)
X_product = X1 * X2

# Inverse FFT to get the time-domain result
x_freq_conv = np.fft.ifft(X_product)

# Plot the results
plt.figure(figsize=(12, 6))
plt.subplot(2, 1, 1)
plt.plot(n, x_conv, label="Time-Domain Convolution")
plt.title("Time-Domain Convolution vs Frequency-Domain Multiplication (Rectangular Pu
plt.grid(True)

plt.subplot(2, 1, 2)
plt.plot(n, np.real(x_freq_conv), label="Frequency-Domain Convolution")
plt.grid(True)

plt.tight_layout()
plt.show()
```

Hasil:



2.7

Program:

```
import torch
import numpy as np

# Frequency-domain representation of the signal (DTFT samples)
omega = torch.linspace(-np.pi, np.pi, 100)  # 100 frequency samples between -pi and pi
X_omega = torch.exp(-1j * omega)  # Example DTFT X(e^jω) = exp(-jω)

# Inverse DTFT: numerical approximation using sum over discrete omega
n = torch.arange(-10, 11)  # Time indices for the output signal x[n]
x_n = torch.zeros(len(n), dtype=torch.cfloat)

for i in range(len(n)):
    x_n[i] = torch.sum(X_omega * torch.exp(1j * omega * n[i])) / len(omega)

# Real part of the result, as the original signal is real-valued
print(x_n.real)
```

Hasil:

```
Reloaded modules: torch.ops, torch.classes
tensor([-0.0100,  0.0100, -0.0100,  0.0100, -0.0100,
0.0100, -0.0100,  0.0100,
        -0.0100,  0.0100, -0.0100,  1.0000, -0.0100,
0.0100, -0.0100,  0.0100,
        -0.0100,  0.0100, -0.0100,  0.0100, -0.0100])
```

Variasi 1, menggunakan fungsi yang berbeda:

```
import torch
import numpy as np

# Frequency-domain representation (using a sinc function)
omega = torch.linspace(-np.pi, np.pi, 100)
X_omega = torch.sinc(omega / np.pi)  # Sinc function in frequency domain

# Inverse DTFT approximation
n = torch.arange(-10, 11)
x_n = torch.zeros(len(n), dtype=torch.cfloat)

for i in range(len(n)):
    x_n[i] = torch.sum(X_omega * torch.exp(1j * omega * n[i])) / len(omega)

# Print real part of reconstructed signal
print(x_n.real)
```

Hasil:

```
Citra/2.7v1.py' --wdir
tensor([-0.0010,  0.0013, -0.0016,  0.0021, -0.0029,
0.0041, -0.0065,  0.0117,
        -0.0280,  0.2235,  0.5836,  0.2235, -0.0280,
0.0117, -0.0065,  0.0041,
        -0.0029,  0.0021, -0.0016,  0.0013, -0.0010])
```

Variasi 2, menggunakan lebih banyak sampel:

```
import torch
import numpy as np

# Frequency-domain representation (same as original)
omega = torch.linspace(-np.pi, np.pi, 1000)  # Increase frequency samples
X_omega = torch.exp(-1j * omega)  # Same function as original

# Inverse DTFT approximation
n = torch.arange(-10, 11)
x_n = torch.zeros(len(n), dtype=torch.cfloat)

for i in range(len(n)):
    x_n[i] = torch.sum(X_omega * torch.exp(1j * omega * n[i])) / len(omega)

# Print real part of reconstructed signal
print(x_n.real)
```

Hasil:

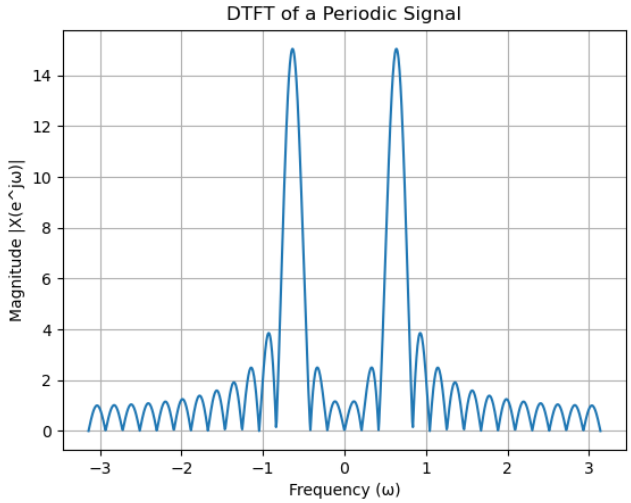
```
tensor([-1.0000e-03,  9.9998e-04, -9.9999e-04,  1.0000e-03,
-1.0000e-03,
         1.0000e-03, -1.0000e-03,  1.0001e-03, -1.0000e-03,
1.0001e-03,
        -1.0000e-03,  1.0000e+00, -1.0000e-03,  1.0001e-03,
-1.0000e-03,
         1.0000e-03, -1.0000e-03,  1.0001e-03, -1.0000e-03,
1.0001e-03,
        -1.0000e-03])
```

2.8

Program:

```
import torch
import numpy as np
import matplotlib.pyplot as plt

# Define the periodic signal (cosine wave)
N = 10  # Period of the signal
n = torch.arange(0, 30)  # Time indices
x_n = torch.cos(2 * np.pi * n / N)  # Periodic signal

# Perform DTFT: Fourier Transform of the periodic signal
omega = torch.linspace(-np.pi, np.pi, 1000)
X_omega = torch.zeros_like(omega, dtype=torch.cfloat)

for i in range(len(omega)):
    X_omega[i] = torch.sum(x_n * torch.exp(-1j * omega[i] * n))

# Plot the magnitude of the DTFT
plt.plot(omega, X_omega.abs().numpy())
plt.title("DTFT of a Periodic Signal")
plt.xlabel("Frequency (ω)")
plt.ylabel("Magnitude |X(e^jω)|")
plt.grid(True)
plt.show()
```

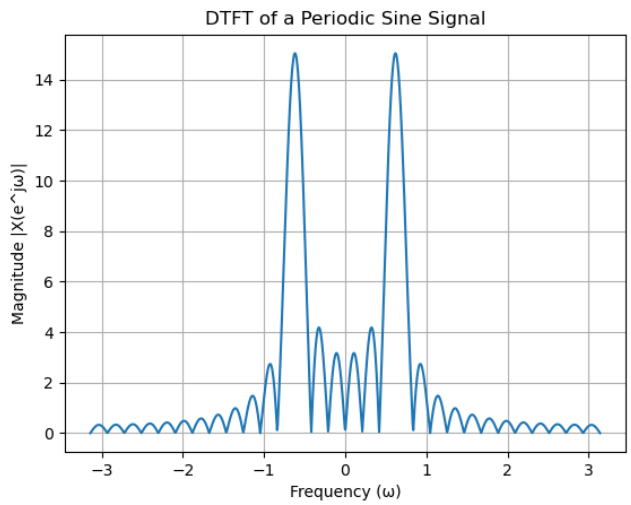
Hasil:



Variasi 1, menggunakan gelombang sinus:

```
import torch
import numpy as np
import matplotlib.pyplot as plt

# Define the periodic signal (sine wave instead of cosine)
N = 10  # Period of the signal
n = torch.arange(0, 30)  # Time indices
x_n = torch.sin(2 * np.pi * n / N)  # Change cosine to sine

# Perform DTFT
omega = torch.linspace(-np.pi, np.pi, 1000)
X_omega = torch.zeros_like(omega, dtype=torch.cfloat)

for i in range(len(omega)):
    X_omega[i] = torch.sum(x_n * torch.exp(-1j * omega[i] * n))

# Plot the magnitude of the DTFT
plt.plot(omega, X_omega.abs().numpy())
plt.title("DTFT of a Periodic Sine Signal")
plt.xlabel("Frequency (ω)")
plt.ylabel("Magnitude |X(e^jω)|")
plt.grid(True)
plt.show()
```

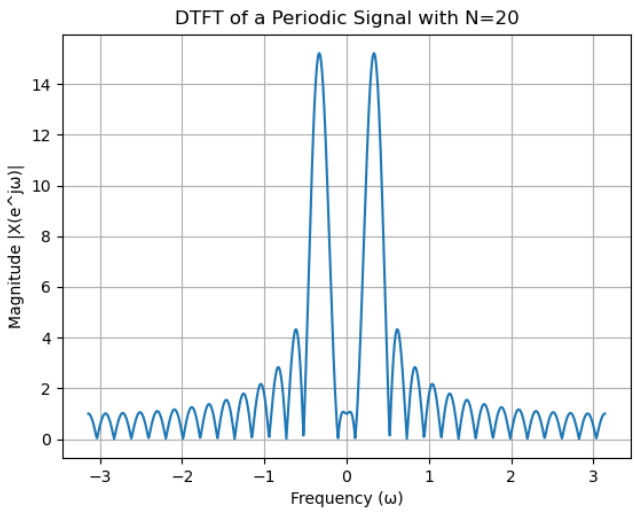
Hasil:



Variasi 2, menggunakan periode yang lebih besar (N=20):

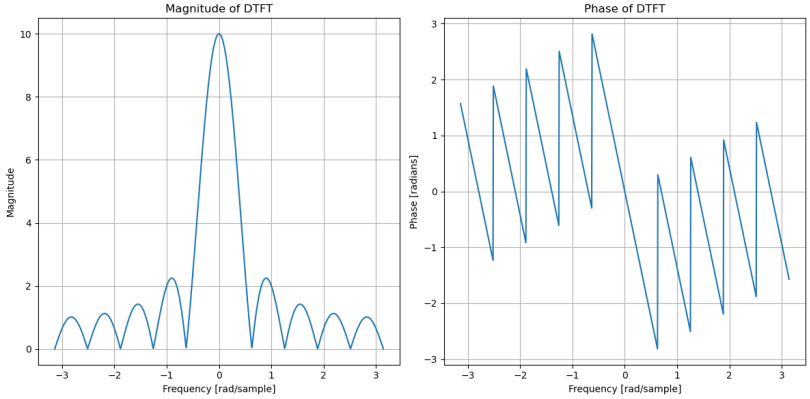
```
import torch
import numpy as np
import matplotlib.pyplot as plt

# Define the periodic signal (cosine wave with different period)
N = 20  # Increased period
n = torch.arange(0, 30)  # Time indices
x_n = torch.cos(2 * np.pi * n / N)  # Same function, different period

# Perform DTFT
omega = torch.linspace(-np.pi, np.pi, 1000)
X_omega = torch.zeros_like(omega, dtype=torch.cfloat)

for i in range(len(omega)):
    X_omega[i] = torch.sum(x_n * torch.exp(-1j * omega[i] * n))

# Plot the magnitude of the DTFT
plt.plot(omega, X_omega.abs().numpy())
plt.title("DTFT of a Periodic Signal with N=20")
plt.xlabel("Frequency (ω)")
plt.ylabel("Magnitude |X(e^jω)|")
plt.grid(True)
plt.show()
```

Hasil:

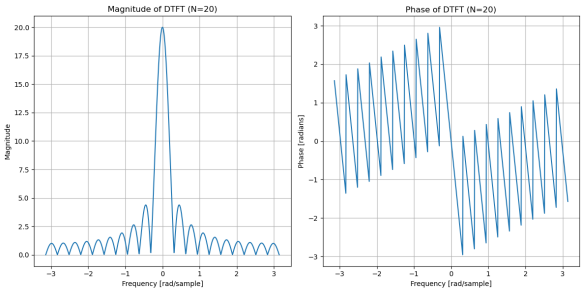


3. ANALISA

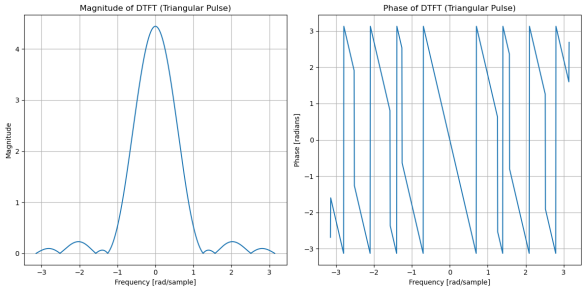
Pada praktikum ini, penerapan DTFT dilakukan untuk menganalisis spektrum frekuensi dari sinyal diskrit [2.4.1]. Program pertama mengambil sinyal rectangular pulse (deret bilangan 1 sepanjang N=10), lalu menghitung DTFT dengan menjumlahkan hasil perkalian sinyal dengan eksponensial kompleks pada berbagai titik frekuensi. Hasilnya ditampilkan dalam dua grafik: magnitudo (yang menunjukkan seberapa kuat setiap frekuensi hadir dalam sinyal) dan fase (yang menggambarkan pergeseran sudut akibat frekuensi tertentu). Dari hasilnya, terlihat sinyal persegi membentuk pola sinc function, menunjukkan bahwa semakin panjang sinyal, semakin tajam puncak frekuensinya.



Pada variasi pertama, panjang sinyal dinaikkan menjadi N=20, menyebabkan puncak spektrumnya menjadi lebih tajam dan lebih sempit dibandingkan dengan sinyal asli. Hal ini sesuai dengan teori bahwa semakin panjang sinyal dalam domain waktu, semakin sempit lebar pita dalam domain frekuensi. Sementara itu, pada variasi kedua, bentuk sinyal diubah menjadi puls triangular menggunakan jendela Bartlett. Hasilnya menunjukkan bahwa dibandingkan dengan sinyal persegi, spektrum magnitudo lebih halus dan tidak sepadat sebelumnya, karena bentuk segitiga memiliki transisi yang lebih bertahap dibandingkan sinyal persegi. Membuktikan perubahan bentuk dan panjang sinyal dalam domain waktu akan berdampak langsung pada karakteristik spektrumnya dalam domain frekuensi.



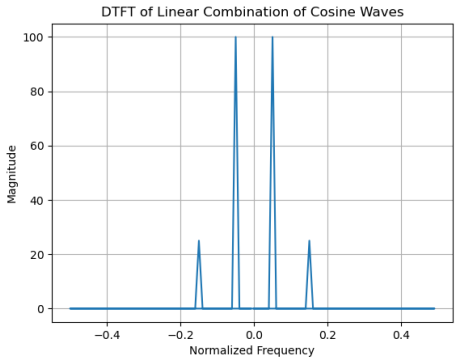
Var. 1



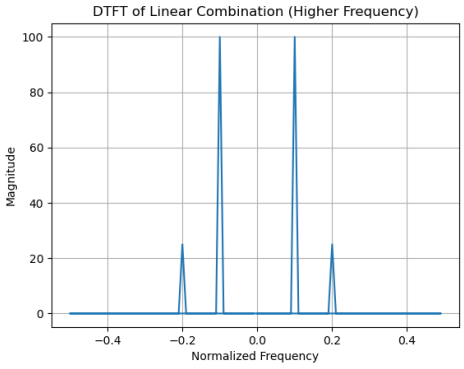
Var. 2

Percobaan selanjutnya penerapan sifat linearitas dengan menganalisis kombinasi linear dari dua sinyal sinusoidal [2.6.1]. Program pertama mendefinisikan dua sinyal kosinus dengan frekuensi berbeda, lalu menggabungkannya menggunakan koefisien tertentu sebelum dihitung DTFT nya menggunakan FFT. Hasilnya menunjukkan spektrum hasil kombinasi linear masih mencerminkan frekuensi dari kedua sinyal penyusunnya, tetapi dengan amplitudo yang dipengaruhi oleh faktor skalar dari kombinasi linear tersebut. Sehingga sifat linearitas tetap

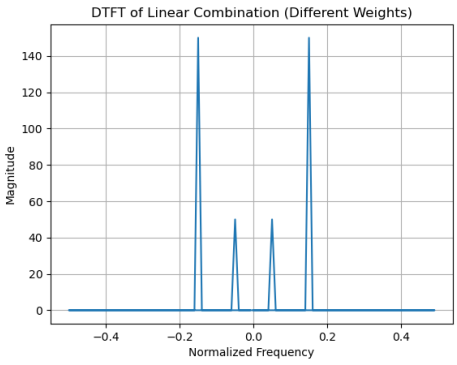
berlaku dalam domain frekuensi, di mana penjumlahan dua sinyal dalam domain waktu akan menghasilkan spektrum yang juga merupakan kombinasi dari spektrum masing-masing sinyal.



Pada variasi pertama, frekuensi kedua sinyal ditingkatkan sehingga spektrum magnitudo bergeser ke frekuensi yang lebih tinggi, tetapi tetap mempertahankan karakteristik gabungan dari dua sinyal sinusoidal. Menunjukkan perubahan frekuensi dalam domain waktu secara langsung memengaruhi lokasi puncak dalam spektrum frekuensi. Pada variasi kedua, koefisien kombinasi linear diubah sehingga sinyal kedua lebih dominan dibandingkan yang pertama. Hasilnya, spektrum menunjukkan peningkatan amplitudo pada frekuensi terkait sinyal kedua, membuktikan bahwa skala dalam domain waktu berpengaruh langsung terhadap amplitudo spektrum frekuensi. Sehingga sifat linear dan perubahan dalam domain waktu terdapat pada DTFT, baik dari segi frekuensi maupun skala, akan tercermin langsung dalam domain frekuensi.

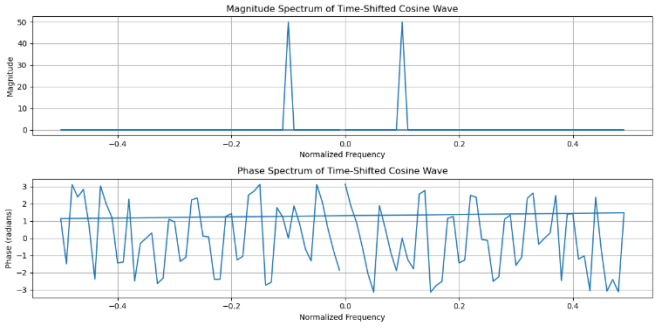


Var. 1

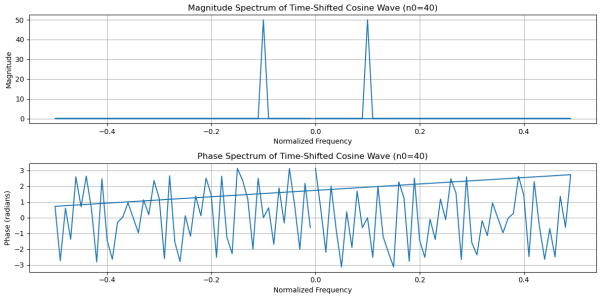


Var. 2

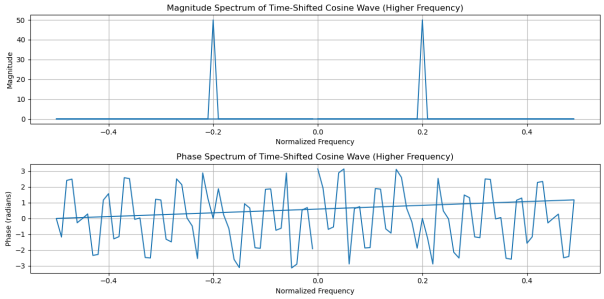
Percobaan selanjutnya penerapan sifat pergeseran waktu dengan menggeser sinyal kosinus dalam domain waktu dan menganalisis pengaruhnya terhadap spektrum frekuensi [2.6.2]. Program pertama mendefinisikan sinyal kosinus, lalu menggesernya sejauh 20 sampel sebelum dihitung DTFT nya menggunakan FFT seperti sebelumnya. Hasilnya menunjukkan pergeseran waktu tidak mengubah magnitudo spektrum frekuensi namun menyebabkan perubahan pada spektrum fase. Sesuai dengan teori bahwa pergeseran waktu dalam domain waktu hanya memengaruhi fase dalam domain frekuensi dengan pergeseran yang berbanding lurus terhadap frekuensi sinyal.



Pada variasi pertama, nilai pergeseran diperbesar menjadi 40 sampel, menunjukkan bahwa semakin besar pergeseran waktu, semakin besar pula perubahan fase dalam domain frekuensi, tetapi magnitudo tetap tidak berubah. Pada variasi kedua, frekuensi sinyal ditingkatkan menjadi 0.4π rad/sampel, menunjukkan bahwa perubahan fase lebih cepat dibandingkan dengan frekuensi yang lebih rendah. Ini menunjukkan bahwa pergeseran waktu lebih berdampak pada sinyal dengan frekuensi tinggi dibandingkan dengan frekuensi rendah. Kesimpulannya, percobaan ini sesuai dengan teori bahwa pergeseran waktu hanya memengaruhi fase dalam domain frekuensi, tanpa mengubah magnitudo spektrumnya.

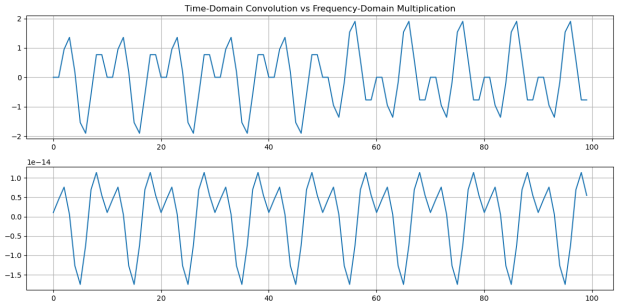


Var. 1

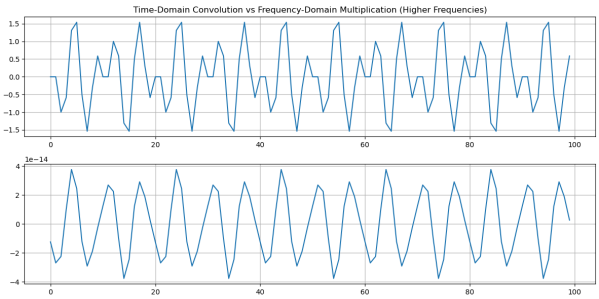


Var. 2

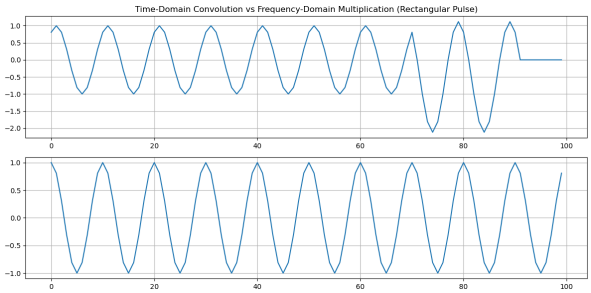
Percobaan selanjutnya membandingkan konvolusi dalam domain waktu dengan perkalian dalam domain frekuensi [2.6.3]. Program pertama melakukan konvolusi dua sinyal, yaitu gelombang kosinus dan gelombang sinus, lalu membandingkannya dengan hasil perkalian spektrum frekuensinya menggunakan FFT seperti sebelumnya. Hasilnya konvolusi dalam domain waktu memiliki bentuk yang mirip dengan hasil transformasi balik dari perkalian dalam domain frekuensi. Ini menunjukkan bahwa konvolusi dalam domain waktu setara dengan perkalian dalam domain frekuensi, seperti yang diprediksi dalam teori sinyal.



Pada variasi pertama, frekuensi sinyal ditingkatkan, dan menunjukkan bahwa sinyal hasil konvolusi memiliki komponen frekuensi yang lebih tinggi dibandingkan dengan percobaan awal, sesuai dengan ekspektasi. Pada variasi kedua, salah satu sinyal diganti dengan pulsa persegi panjang, menyebabkan hasil konvolusi memiliki pola yang lebih terlokalisasi dalam domain waktu. Sehingga menguatkan konsep bahwa operasi konvolusi dalam domain waktu memiliki keterkaitan langsung dengan perkalian dalam domain frekuensi.



Var. 1



Var. 2

Percobaan selanjutnya membahas proses inversi melalui pendekatan numerik [2.7]. Program pertama mengambil sampel DTFT dari sinyal $X(e^{j\omega})e^{-j\omega}$, lalu menghitung kembali sinyal dalam domain waktu dengan menjumlahkan kontribusi dari setiap frekuensi. Hasil dari sinyal yang direkonstruksi memiliki pola yang sesuai dengan ekspektasi, yaitu nilai dominan di n=0 dan nilai kecil untuk indeks lainnya. Menunjukkan metode inversi yang digunakan cukup akurat dalam merekonstruksi sinyal dari domain frekuensi ke domain waktu.

```
Reloaded modules: torch.ops, torch.classes
tensor([-0.0100,  0.0100, -0.0100,  0.0100, -0.0100,
0.0100, -0.0100,  0.0100,
        -0.0100,  0.0100, -0.0100,  1.0000, -0.0100,
0.0100, -0.0100,  0.0100,
        -0.0100,  0.0100, -0.0100,  0.0100, -0.0100])
```

Pada variasi pertama, bentuk X(ejω) diubah menjadi fungsi sinc di domain frekuensi. Hasilnya bentuk yang lebih halus dengan nilai-nilai lebih signifikan di sekitar n=0, sesuai dengan sifat transformasi sinc dalam domain waktu. Selanjutnya pada variasi kedua, jumlah sampel frekuensi ditingkatkan menjadi 1000 untuk melihat pengaruh resolusi lebih tinggi. Hasilnya hampir identik dengan percobaan awal, tetapi sedikit lebih presisi karena efek aliasing lebih kecil. Secara keseluruhan, hasil percobaan ini sesuai dengan teori bahwa peningkatan resolusi frekuensi memberikan hasil rekonstruksi yang lebih akurat, dan bentuk spektrum frekuensi sangat memengaruhi bentuk sinyal di domain waktu.

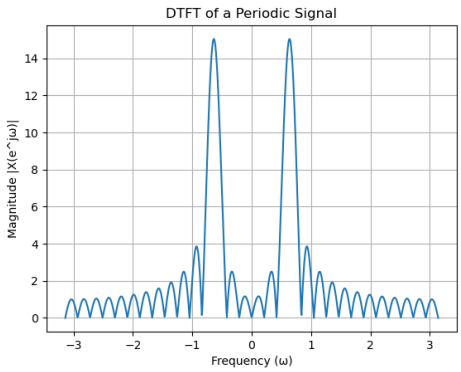
```
Citra/2.7v1.py' --wdir
tensor([-0.0010,  0.0013, -0.0016,  0.0021, -0.0029,
0.0041, -0.0065,  0.0117,
        -0.0280,  0.2235,  0.5836,  0.2235, -0.0280,
0.0117, -0.0065,  0.0041,
        -0.0029,  0.0021, -0.0016,  0.0013, -0.0010])
```

Var. 1

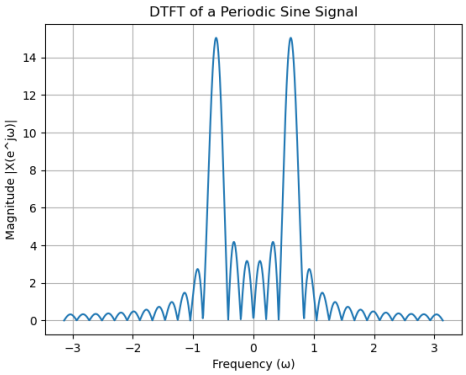
```
tensor([-1.0000e-03,  9.9998e-04, -9.9999e-04,  1.0000e-03,
-1.0000e-03,
         1.0000e-03, -1.0000e-03,  1.0001e-03, -1.0000e-03,
1.0001e-03,
        -1.0000e-03,  1.0000e+00, -1.0000e-03,  1.0001e-03,
-1.0000e-03,
         1.0000e-03, -1.0000e-03,  1.0001e-03, -1.0000e-03,
1.0001e-03,
        -1.0000e-03])
```

Var. 2

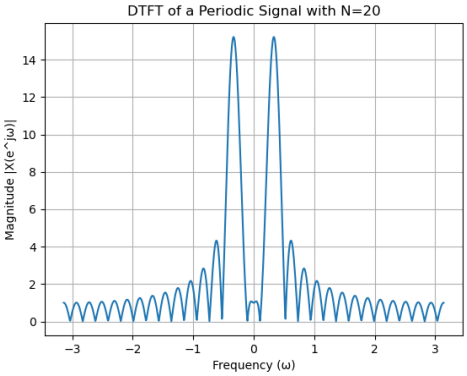
Percobaan selanjutnya representasi dari sinyal periodik dalam domain frekuensi [2.8]. Program pertama menggunakan gelombang kosinus dengan periode N=10, kemudian dihitung dengan menjumlahkan kontribusi dari setiap titik waktu. Hasilnya adalah grafik magnitudo DTFT yang menunjukkan puncak-puncak pada frekuensi tertentu, mencerminkan sifat periodik dari sinyal input. Sehingga sinyal periodik dalam domain waktu akan memiliki spektrum frekuensi dengan komponen yang jelas pada harmonik fundamentalnya.



Pada variasi pertama, sinyal yang digunakan diubah dari kosinus menjadi sinus. Mnyebabkan perbedaan pada distribusi fase dalam domain frekuensi, tetapi pola magnitudo DTFT tetap menunjukkan puncak pada lokasi yang sama, karena sifat dasar sinus dan kosinus yang hanya berbeda fasa. Variasi kedua mengubah periode sinyal menjadi N=20, menyebabkan puncak-puncak spektrum menjadi lebih rapat, sesuai dengan teori bahwa semakin besar periodenya, semakin dekat jarak antar komponen frekuensinya. Menunjukkan adanya hubungan erat antara sifat periodik suatu sinyal dan representasi frekuensinya melalui DTFT.



Var. 1



Var. 2

4. KESIMPULAN

Penerapan DTFT dalam praktikum ini berhasil menunjukkan bagaimana karakteristik sinyal dalam domain waktu berpengaruh langsung terhadap spektrumnya dalam domain frekuensi. Semakin panjang sinyal, semakin tajam dan sempit spektrum frekuensinya, sementara bentuk sinyal juga memengaruhi kehalusan spektrum. Percobaan sifat linearitas membuktikan bahwa kombinasi dua sinyal dalam domain waktu tetap tercermin dalam spektrum gabungan di domain frekuensi. Pergeseran waktu hanya mengubah fase tanpa memengaruhi magnitudo, sedangkan konvolusi dalam domain waktu setara dengan perkalian dalam domain frekuensi. Selain itu, metode inversi DTFT berhasil merekonstruksi sinyal dengan cukup akurat, terutama dengan peningkatan resolusi frekuensi. Terakhir, percobaan dengan sinyal periodik menunjukkan bahwa sinyal yang berulang dalam domain waktu akan menghasilkan spektrum dengan puncak-puncak pada harmonik fundamentalnya. Kesimpulannya setiap perubahan dalam domain waktu memiliki dampak langsung pada domain frekuensi.